

CAPACITY CONNECT

A Digital Capacity Building & Learning Management Portal

Complete Project Report

Problem Statement ID: SIH25075 | Organization: Ministry of Earth Sciences (MoES)

Category: Software | Theme: Smart Education

This document consolidates the full solution design: user roles & permissions, required features mapped to the official problem statement, the community/forum module, database and tech-stack decisions, and the recommended build sequence.

1. Problem Statement Summary

The Ministry of Earth Sciences (MoES) requires a centralized digital platform for organizational training, competency development, and knowledge sharing. Participants are asked to design and develop **CAPACITY CONNECT**, a Digital Capacity Building and Learning Management Portal that supports structured training delivery, skill tracking, and certification across the organization.

The official description requires: secure signup/login for three roles (Trainee, Trainer, Admin); trainee profiles with qualifications, experience, interests, skills and certificates; course enrollment, learning resources, subject-wise MCQ assessments and feedback; trainer profile management, questionnaires with deadlines, participation/performance monitoring, and a trainer library for lectures, presentations and study material; an admin module for user approval, role management, dashboards (courses, enrollments, certifications, assessments, participation) and homepage publishing of notifications/announcements/achievements/new content; and organization-wide competency mapping to identify suitable trainers per subject. The platform must be scalable, secure, user-friendly and accessible across devices.

2. Proposed Solution Overview

Capacity Connect is a role-based LMS with an integrated community layer. Every course automatically gets a linked discussion space ("course-forum"), combining open Reddit-style discussion with an optional Q&A mode (accepted-answer marking) so that knowledge sharing feels informal but troubleshooting still gets resolved. A trainee-facing portfolio and skill-progress view turns certifications and course history into a visible competency record, which directly powers the ministry's competency-mapping requirement (matching trainers to subjects, and trainees to skill gaps).

Analogy used for the design: the **LMS is the classroom** (structured teaching, assessment, certification) and the **Forum is the classroom community** (peer support, doubt-clearing, informal knowledge exchange) — fully optional, but proven to raise engagement and retention.

3. User Roles Overview

Role	Who they are	Primary goal on the platform
Trainee	Employee/staff undergoing training	Learn, get certified, build a visible skill profile
Trainer	Subject-matter expert delivering training	Create/deliver courses, assess trainees, build teaching credibility
Admin	Ministry/platform administrator	Govern the platform: approve users, monitor stats, publish content
Visitor	Not-yet-registered / unauthenticated user	Preview public content before signing up

4. Complete Roles & Permissions Matrix

Legend: **Full** = create/edit/delete, **Own** = only their own content, **View** = read-only, **—** = no access.

Capability	Trainee	Trainer	Admin	Visitor
Sign up / secure login (RBAC)	Full	Full	Full	—
Build own profile (qualifications, experience, skills, interests)	Own	Own	View all	—

Capability	Trainee	Trainer	Admin	Visitor
Browse course catalog	View	View	View	View
Enroll in a course	Full	—	View	—
Access learning resources of enrolled courses	View	Own courses	View all	—
Attempt MCQ assessments / exams	Full	—	View results	—
Give course/content feedback	Full	View own	View all	—
Create / edit a course	—	Own	Approve/Reject	—
Create quizzes/questionnaires with deadlines	—	Own	View	—
Upload lectures / slides / study material (Trainer Library)	—	Own	Moderate	—
Monitor trainee participation & performance	Own progress	Own courses	All	—
Issue / validate certificates	Receive	Recommend	Validate/Approve	—
Approve new user sign-ups	—	—	Full	—
Manage roles (promote/demote/suspend)	—	—	Full	—
View platform-wide dashboards (enrollment, certs, assessments)	—	Own courses only	Full	—
Publish homepage notifications / announcements / achievements	—	Request	Full	View
Competency mapping (trainer-subject matching)	View own	View own	Full / configure	—
Delete a user profile	—	—	Full	—
Post a discussion / question thread in a course forum	Full	Full	Full	View only
Reply to a thread	Full	Full	Full	—
Mark a reply as accepted answer	Own thread	Own course threads	Any	—
Upvote posts/replies	Full	Full	Full	—
Report / flag content	Full	Full	Full	—
Moderate reported content (hide/remove/warn)	—	Own course forum	Full, any forum	—

5. Detailed Feature List — What Each User Can Do

5.1 Trainee — Features

Required (per problem statement):

- Secure signup/login
- Professional profile: qualifications, work experience, interests, skills, certificates
- Browse and enroll in courses
- Access learning resources (videos, PDFs, slides) for enrolled courses
- Attempt subject-wise MCQ assessments and final exams
- Give feedback/ratings on courses and training content

Differentiators we are adding:

- Portfolio page: auto-built from completed courses, skills and certificates — a shareable competency record
- Skill-gap tracker: visual comparison of current skills vs. role-required skills, with suggested courses
- Downloadable/verifiable certificates (QR code or unique hash for authenticity checking)
- Personalized course recommendations based on role/department and skill gaps
- Leaderboards / badges (per department or organization-wide) for motivation
- Participate in course-linked discussion forums ("subreddits") — ask questions, discuss, help peers
- In-app notifications: deadline reminders, certificate expiry, reply-to-my-question alerts
- Offline / low-bandwidth mode for field staff (ships, remote stations) with poor connectivity

5.2 Trainer — Features

Required (per problem statement):

- Manage own profile
- Create and publish courses
- Create questionnaires/quizzes with deadlines
- Monitor trainee participation and performance
- Upload recorded lectures, presentations and study material to a Trainer Library accessible to trainees

Differentiators we are adding:

- Public trainer profile showing experience, total students taught, and certifications provided
- Course analytics dashboard: drop-off rate, quiz score distribution, average completion time
- Reusable question bank shared across a trainer's own courses
- Verified-trainer badge (visible on forum replies and course listings) once approved by Admin
- Ability to mark a forum reply as the "accepted answer" in their own course's discussion space
- Feedback loop: aggregated trainee ratings feed into the trainer's public profile

5.3 Admin — Features

Required (per problem statement):

- Approve/reject new user signups and manage roles
- View/delete trainer and trainee profiles
- Validate trainer credentials and validate courses/certifications before they go live

- Dashboards: courses, enrollments, certifications, assessments, participation statistics
- Publish notifications, announcements, achievements and newly added learning content on the homepage

Differentiators we are adding:

- Competency mapping tool: identify the best-fit trainer for a given subject based on verified skills/history
- Audit logs: who approved/rejected what, and when (accountability, required for a govt platform)
- Bulk import of trainees via employee ID/department CSV (HR-system integration)
- Organization-wide analytics: which departments are under-trained, mandatory-course compliance tracking
- Export reports (CSV/PDF) for compliance reporting up the ministry chain
- Moderation queue for reported forum content across all course forums

5.4 Visitor — Features (unauthenticated)

- View the public homepage: announcements, achievements, newly added courses
- Browse the course catalog and read course descriptions (read-only)
- View course forums in read-only mode (cannot post, reply or vote)
- Prompted to sign up to enroll, post, or track progress

6. Community / Forum Module

A Reddit-style, informal discussion model is used instead of a strictly structured Q&A format, because knowledge-sharing works best when it doesn't feel like a formal test. Moderation is kept reasonable rather than heavy-handed: clear boundaries against illegal content, harassment and spam, while allowing informal tone and disagreement — the natural texture of real knowledge-sharing. For a government-facing platform, the recommended default is to start moderately strict (profanity filter, auto-hide after N reports, no anonymous posting) and relax settings as trust in the community builds, rather than the reverse.

6.1 Key Design Decisions

- **Discussion + Q&A combined:** any thread can optionally be marked a "question," enabling an accepted-answer marker on one reply, while all other threads remain open discussion.
- **Course-linked forums ("subreddits"):** every course auto-generates its own discussion space with no manual setup, filterable by skill-tag badges — doubling as a course-discovery tool.
- **Classroom analogy:** the LMS is the actual learning (teacher teaches, student learns); the forum is the classroom community — completely optional, but helps quieter or less confident learners find their footing and stay motivated.
- **Voting:** upvote/downvote for sorting the best answers to the top. For an organizational context, consider upvote-only (no public downvotes) to avoid peer-pressure dogpiling on trainer/trainee replies.

6.2 Forum Feature Checklist

Feature	Notes / Decision
Voting / sorting	Upvote (and optionally downvote) on posts/replies; sort by "best", "new", "top".
Pagination / infinite scroll	Required once a course forum has more than a handful of threads.
Reporting / flagging	Report button feeding a moderation queue for Trainer (own course) / Admin (any forum).
Visitor access	Read-only: can view threads/replies, cannot post, reply, or vote.
Empty-state handling	"Be the first to post" prompt for new/small course forums (cold-start problem).
Media validation	File type/size checks + NSFW/image-content filter on uploads, since forum content is user-generated at scale.
Notifications	At minimum an in-app "unread replies" indicator when a trainee's question gets a reply.
Seed / demo data	Realistic fake threads/replies/votes generated ahead of time so the forum looks alive for judging.
Edit/delete history	Edited posts show "(edited)"; deleted posts are soft-deleted (kept as "[deleted]" placeholder) so reply threads never orphan.
Nested reply depth	Capped depth (recommended 2-3 levels) to keep threads readable.
Accepted-answer permission	Settable by the original poster, or by the course's trainer for extra authority.
Verified-trainer badge	Trainer-verification status (from Admin approval) shown on forum replies for trust signalling.

7. Must-Have Feature Checklist (Mapped to Official Requirement)

Feature	Type	Belongs To
Secure signup/login, 3 roles (+ Visitor)	Core	Auth + RBAC middleware at API layer
Trainee profile: qualifications, experience, interests, skills, certificates	Core	Profile module
Course enrollment + learning resources	Core	Course/Enrollment module
Subject-wise MCQ assessments	Core	Assessment module
Course/content feedback	Core	Feedback module
Trainer profile management	Core	Profile module
Questionnaires with deadlines	Core	Assessment module (deadline field)
Monitor trainee participation/performance	Core	Trainer analytics
Trainer Library (lectures/slides/material upload)	Core	Content/Media module
Admin: user approval + role management	Core	Admin module
Admin: dashboards (courses, enrollments, certs, assessments, participation)	Core	Admin analytics
Admin: homepage publishing (notifications/announcements/achievements/content)	Core	CMS/Notification module
Competency mapping (trainer-subject matching)	Core	Competency module
Scalable / secure / cross-device	Core (NFR)	Architecture + responsive frontend
Portfolio + skill-gap tracking	Differentiator	Portfolio module
Course-linked discussion forums	Differentiator	Forum module
Verified-trainer badges	Differentiator	Ties Admin validation to Forum
Verifiable certificates (QR/hash)	Differentiator	Certificate module
Offline / low-bandwidth mode	Differentiator	PWA / caching layer
Audit logs + compliance export	Differentiator	Admin module

8. Database Schema — Core Entities

A relational database (PostgreSQL) is recommended: posts, replies, users, courses, votes and permissions are all relationally linked (foreign keys such as post→course, reply→post, user→role), and joins like "show all posts by users enrolled in course X" stay simple. NoSQL is avoided because nested replies and access-control joins get messy in a document store.

Table	Key Fields
users	id, name, email, password_hash, role (trainee/trainer/admin), status (pending/approved), profile fields
courses	id, title, description, trainer_id, tags/skills, status (draft/pending/published)
enrollments	id, user_id, course_id, enrolled_at, progress, completion_status
assessments	id, course_id, type (quiz/exam), deadline, questions[]
attempts	id, assessment_id, user_id, score, submitted_at
certificates	id, user_id, course_id, issued_at, verification_hash, status (auto/admin-validated)
resources	id, course_id, type (video/pdf/slide), url, uploaded_by
feedback	id, user_id, course_id, rating, comment
forum_threads	id, course_id, author_id, title, body, is_question, accepted_reply_id
forum_replies	id, thread_id, parent_reply_id (nesting), author_id, body, votes, is_deleted
reports	id, content_type, content_id, reported_by, reason, status
notifications	id, user_id, type, message, read_status
competency_map	id, trainer_id, skill_tag, verified_by_admin (bool)

9. Recommended Tech Stack

Concern	Suggested Tech	Why
Backend / API	Node.js + Express (or Django, matched to whichever stack the team already knows)	Confirm this once — the forum must consume the same stack/auth as the LMS to avoid duplicate systems.
Database	PostgreSQL (relational)	Handles relational joins (post→course, reply→post, user→role) cleanly; avoids NoSQL nested-reply mess.
Auth	JWT-based sessions, shared across LMS and Forum	One login, one source of truth for roles — directly serves the RBAC requirement.
File / media storage	Local disk for the hackathon demo; note AWS S3 (or equivalent) as the production answer	Avoid building cloud storage integration under time pressure; keep validation (type/size) either way.
Real-time updates	Skip for the hackathon; mention WebSockets/Socket.io as future scope	Live-updating threads are a nice-to-have, not core — polling or refresh-on-action is fine for a demo.

Concern	Suggested Tech	Why
Search	Basic keyword/tag search (SQL LIKE / full-text) to start; mention embedding-based semantic search as a stretch goal	Full semantic search is a real scope commitment — don't let it block the MVP.

10. Recommended Build Sequence

1. Database schema design (all of it, upfront)

Users, roles, courses, posts, replies, votes, tags, media attachments. Do this before any interface code — changing it later cascades into every other layer.

2. Access specifiers & permissions (backend only, no UI)

Define what each role (trainer/trainee/admin/visitor) can do at the data/API level, as middleware/guards, so every endpoint respects roles from day one.

3. Core LMS + Forum backend (API only)

CRUD for courses, enrollments, assessments; CRUD for threads/replies, the question-flag + accepted-answer mechanic, capped nested-reply depth.

4. Cross-module integration layer

Course list, tags, enrollment data and trainer-verification status flow between the LMS and Forum (shared DB or internal API); auto-create a forum space when a course is created.

5. Media/resource handling (backend)

Upload endpoint, type/size validation, storage — built after core objects (courses, posts) already exist to attach to.

6. Cross-linking

Deep-links from forum threads back to specific courses/trainers, and from courses to their discussion space.

7. Frontend / UI-UX (last)

By this point the API is stable and permission-aware, so the frontend is wiring up calls to already-correct logic, not discovering logic gaps through the UI.

11. Security & Non-Functional Requirements

- Role-based access control (RBAC) enforced at the API layer, not just the UI
- Password hashing (bcrypt/argon2); JWT-based sessions with expiry and refresh tokens
- Two-factor authentication recommended for Admin accounts
- Encrypted storage for personal data and certificates
- Input validation and file-type/size checks on every upload endpoint
- Audit logs for all admin approval/rejection/deletion actions
- Responsive, cross-device UI; offline/low-bandwidth support for field deployments

12. Conclusion

Capacity Connect satisfies every mandatory requirement in the MoES problem statement (three-role system, profiles, enrollment, assessments, feedback, trainer library, admin governance dashboards, homepage publishing, competency mapping) while adding a set of differentiators — the course-linked community forum, trainee portfolios with skill-gap tracking, verifiable certificates, and compliance-grade admin tooling — that make the platform feel like a genuine, ministry-wide capacity building system rather than a bare-bones LMS.